



面向大模型 RAG 推理场景 噪音文档的鲁棒性推理方法

NLP 实验四 • 题目 8 • 中期检查报告

队长姓名：待填写 队长学号：待填写

工作区：D:\code\nlpexp4

2026 年 5 月 25 日

目录

1	研究问题与项目定位	3
1.1	题目要求	3
1.2	核心问题	3
1.3	阶段性结论	3
2	数据准备与标准格式	4
2.1	原始数据	4
2.2	文档池定义	4
2.3	课程要求格式对齐	5
3	系统设计	5
3.1	总体架构	5
3.2	调用流水线	6
3.3	代码结构	6
4	方法设计	6
4.1	噪音注入机制	6
4.2	矫正方法	7
5	评估指标设计	7
5.1	基础问答指标	7
5.2	自主鲁棒性指标	8
6	实验设计	8
6.1	实验一：噪音影响	8
6.2	实验二：矫正机制对比	8
6.3	实验三：案例分析	8
6.4	实验四：现有方法比较	8
7	阶段性实验结果	9
7.1	噪音影响	9
7.2	噪音位置效应	10
7.3	矫正方法结果	10
7.4	现有方法比较	11
7.5	案例类型分布	12
8	系统实现与前端演示	12
8.1	后端接口	12

8.2 前端界面	12
9 当前进度	13
10 问题、边界与后续计划	13
10.1 已识别问题	13
10.2 后续计划	14
11 运行方式	14
12 中期总结	15

1 研究问题与项目定位

1.1 题目要求

本项目对应 NLP 实验四题目 8：面向大模型 RAG 推理场景噪音文档的鲁棒性推理方法。题目要求关注大模型 RAG 场景中，检索结果可能包含与问题语义相关、但缺少逻辑依赖或会误导答案的上下文文档，进而影响模型问答准确率。项目需要完成以下任务：

- 探究上下文文档对模型问答性能的影响；
- 提出可量化的评估指标；
- 针对具体数据集给出实验结果、结论与分析；
- 结合具体案例分析不同文档对答案的影响；
- 设计矫正机制，例如提示词优化、迭代式生成等；
- 与现有方法进行性能比较，说明方法有效性与局限。

1.2 核心问题

本项目将问题形式化为：给定问题 q 、候选文档集合 $D = \{d_1, \dots, d_n\}$ 和参考答案 a ，当 D 中混入不同类型、比例和位置的噪音文档时，大模型生成答案 $\hat{a}$ 的准确率、证据来源和噪音采纳程度如何变化？

实验重点不是重新训练大模型，而是构造可控 RAG 上下文，并研究模型在噪音上下文下的鲁棒推理行为。当前系统采用 DeepSeek API 作为生成模型，以 RGB 数据集中的文档池模拟检索结果。

1.3 阶段性结论

截至中期检查，项目已经完成数据加载、噪音注入、RAG 推理、矫正方法、评估指标、实验脚本、图表生成和交互式前端的主体实现。最新实验显示：

- 高比例噪音会显著降低问答性能，且答案信息来源会从有效文档迁移到噪音文档；
- 中等强度语义噪音不一定总是有害，部分样本中会提供主题线索；
- 在 zh/main semantic r=0.5 条件下，iterative filter-then-generate 方法当前表现最好；
- Self-RAG-style baseline 在当前数据集上判定较宽，效果接近 naive baseline。

2 数据准备与标准格式

2.1 原始数据

项目使用 RGB 数据集作为公开问答与候选文档来源。数据位于 `data/rgb/`，包含中英文主数据、反事实数据、信息整合数据和 refine 数据：

表 1: RGB 数据文件与用途

数据文件	语言	样本数	用途
zh.json, en.json	中/英	各 300	主问答数据，含 positive 与 negative 文档池
zh_fact.json, en_fact.json	中/英	各 100	反事实噪音数据，额外含 positive_wrong 和 fakeanswer
zh_int.json, en_int.json	中/英	各 100	信息整合类问题，用于后续跨文档推理扩展
zh_refine.json, en_refine.json	中/英	各 300	refine 变体，用于后续泛化验证

2.2 文档池定义

RGB 数据天然给出了适合本题的三类文档池：

表 2: 三类文档池与噪音含义

字段	实验含义	使用方式
positive	支持正确答案的有效文档	构造 clean RAG 上下文，计算 ISR
negative	语义相关但不能支持答案的普通噪音文档	模拟检索噪音，计算 NAR
positive_wrong	表面相关但指向错误答案的反事实文档	模拟更强误导性噪音，测试矫正机制

因此，项目没有重新爬虫或解析 PDF，而是采用公开数据集的已标注文档池作为可控检索结果。这样可以精确控制噪音比例、类型和位置，避免真实检索系统中召回误差与生成误差互相混杂。

2.3 课程要求格式对齐

为对齐实验要求，项目已增加标准格式导出脚本 `scripts/export_standard_io.py`，可从实验结果中导出：

- `input.json`: 包含 `id/question/context`，并额外保留文档与标签；
- `output.json`: 包含 `id/answer`，默认导出当前 token-F1 最高方法；
- `reference.json`: 包含 `id/answer/answers`，与输入和输出严格对齐；
- `output_all_methods.json`: 保留所有方法的横向比较输出。

当前标准文件已导出至：

`data/processed/exp4_existing_methods_zh_main_semantic_20260524_200425/`

3 系统设计

3.1 总体架构

系统由数据层、噪音构造层、推理与矫正层、评估层、可视化与前端层组成。其核心思想是：先从 RGB 数据集中读取问题、答案和文档池，再按实验条件注入噪音，最后调用不同 RAG/矫正方法生成答案并计算指标。

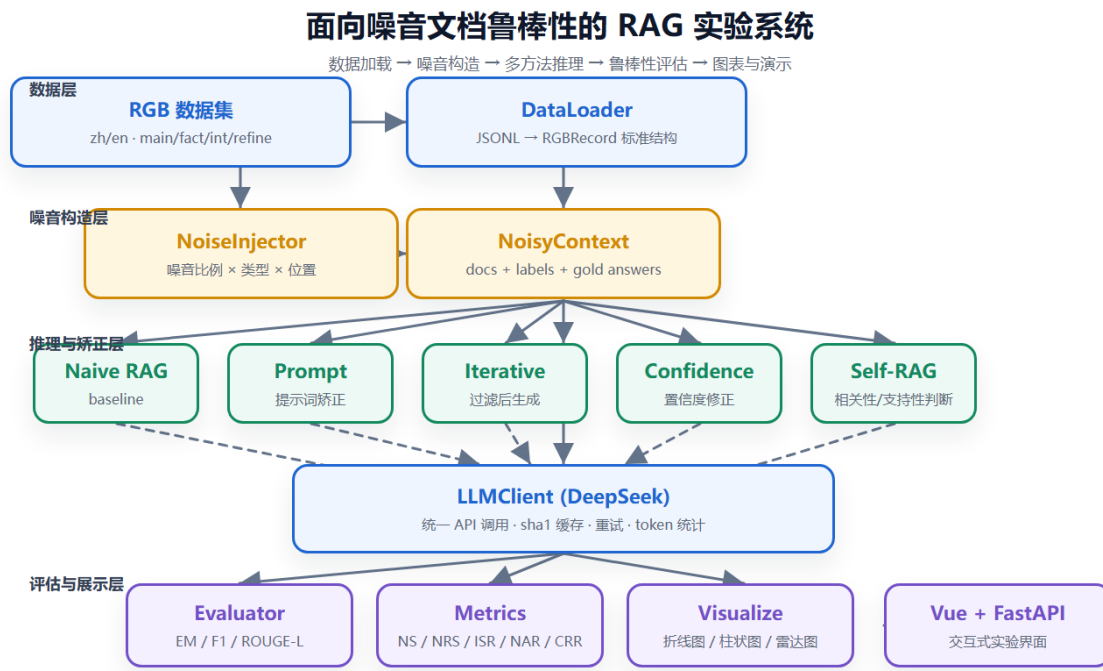


图 1: 系统总体框架图

3.2 调用流水线

单次实验的主流程如下：

1. 加载数据：调用 `load_dataset(language, subset, limit)` 读取 RGB 样本；
2. 注入噪音：调用 `batch_inject` 按噪音比例、类型、位置构造 `NoisyContext`；
3. 选择方法：根据实验条件选择 `naive`、`prompt`、`iterative`、`confidence`、`selfrag` 或 `voting`；
4. 调用模型：通过 `LLMClient` 访问 DeepSeek API，使用缓存减少重复调用；
5. 计算指标：通过 `Evaluator` 和 `metrics.py` 输出基础问答指标与鲁棒性指标；
6. 落盘与可视化：保存 JSON，生成图表，并同步报告与前端展示。

3.3 代码结构

```
D:\code\nlpexp4
|-- data/rgb/                RGB 原始数据
|-- data/processed/         input/output/reference 标准 JSON
|-- src/
|   |-- data_loader.py      RGB 加载与标准化
|   |-- noise_injector.py   噪音比例/类型/位置控制
|   |-- rag_pipeline.py     Naive RAG baseline
|   |-- llm_client.py       DeepSeek API + 缓存 + 重试
|   |-- evaluator.py        EM/F1/ROUGE/LLM-Judge
|   |-- metrics.py          NS/NRS/ISR/NAR/CRR
|   |-- correctors/         prompt/iterative/confidence/selfrag/voting
|-- experiments/
|   |-- exp1_noise_impact.py
|   |-- exp2_correction.py
|   |-- exp3_case_study.py
|   |-- exp4_existing_methods.py
|-- backend/                 FastAPI 接口
|-- frontend/                Vue 3 实验界面
|-- figures/                 实验图表
|-- report_latex/            本 LaTeX 报告
```

4 方法设计

4.1 噪音注入机制

设最大上下文文档数为 K ，目标噪音比例为 r 。系统从 `positive` 中采样 $K(1-r)$ 个有效文档，从噪音池中采样 Kr 个噪音文档。噪音类型包括：

- **semantic**: 从 `negative` 中采样普通语义噪音;
- **counterfactual**: 从 `positive_wrong` 中采样反事实噪音;
- **mixed**: 同时混合普通噪音和反事实噪音。

噪音位置包括 `front`、`back`、`interleave`、`surround`，用于观察上下文位置偏见对回答的影响。

4.2 矫正方法

表 3: 已实现方法

方法	核心思想	API 成本	当前定位
naive	直接拼接文档生成答案	1	基线
prompt	在提示词中要求识别噪音和依据可靠证据回答	1	低成本主方法
iterative	先逐文档判断相关性，再基于保留文档生成	$n + 1$	当前效果最好
confidence	先生成证据链和置信度，再进行答案修正	约 1-2	推理链矫正
selfrag	简化复现 Self-RAG 的相关性与支持性判断	$n + 2$	现有方法对比
voting	多 persona 输出后投票聚合	3+	反事实噪音候选方法

5 评估指标设计

5.1 基础问答指标

基础指标用于判断答案是否接近参考答案:

- **EM**: 规范化后完全匹配;
- **Contains**: 参考答案是否被预测答案包含;
- **Token-F1**: token 级精确率与召回率调和平均;
- **ROUGE-L**: 基于最长公共子序列的 F1;
- **LLM-as-Judge**: 由评审模型给出 1-5 分，当前默认关闭以节约成本。

5.2 自主鲁棒性指标

为刻画噪音文档对模型推理的影响，项目提出 5 个鲁棒性指标：

表 4: 鲁棒性指标体系

指标	定义与含义
NS	Noise Sensitivity, $NS = (S_{clean} - S_{noisy})/S_{clean}$, 衡量噪音导致的相对性能下降
NRS	Noise Resistance Slope, 性能随噪音比例变化的线性斜率, 越接近 0 越稳健
ISR	Info-Source Rate, 答案有效片段可溯源至 positive 文档的比例
NAR	Noise Adoption Rate, 答案有效片段来自 negative 或 positive_wrong 文档的比例
CRR	Correction Recovery Rate, 矫正方法相对 noisy baseline 恢复了多少性能损失

6 实验设计

6.1 实验一：噪音影响

实验一固定 naive RAG, 改变噪音比例、类型和位置, 观察性能曲线。主实验使用 zh/main, 噪音比例为 0、0.25、0.5、0.75、1.0, 噪音类型为 semantic、counterfactual、mixed。

6.2 实验二：矫正机制对比

实验二比较 naive、prompt、confidence 等矫正方法在不同噪音强度下的表现, 重点观察矫正前后 Token-F1、ISR 和 NAR 的变化。

6.3 实验三：案例分析

实验三抽取成功修正、失败修正、无变化等典型案例, 记录 query、gold、预测答案、文档标签与 ISR/NAR, 用于说明不同文档如何影响最终答案。

6.4 实验四：现有方法比较

实验四横向比较 naive、selfrag、iterative、confidence、prompt、voting 六种方法。最新主实验为 zh/main semantic r=0.5, 样本量为 50。

7 阶段性实验结果

7.1 噪音影响

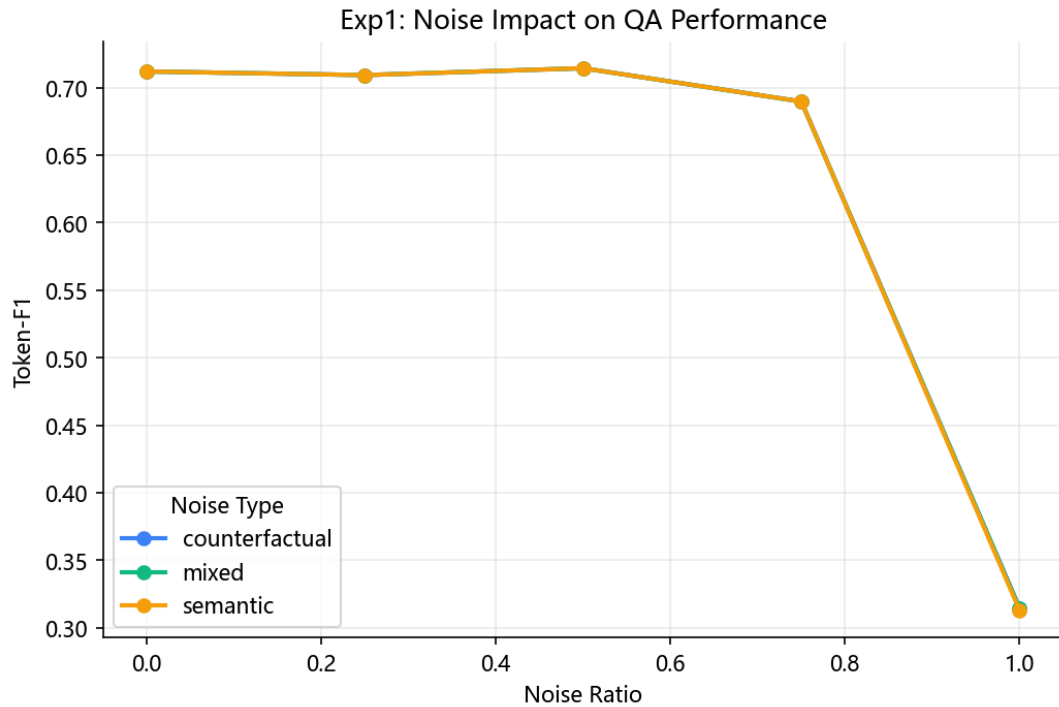


图 2: 实验一：不同噪音比例与类型下的性能变化

实验一显示，高比例噪音会显著破坏 RAG 问答性能。最新主实验中，Token-F1 从约 0.712 下降到约 0.313，Contains 从 0.92 下降到 0.24；同时 NAR 上升，说明模型答案中的有效信息来源明显向噪音文档迁移。

7.2 噪音位置效应

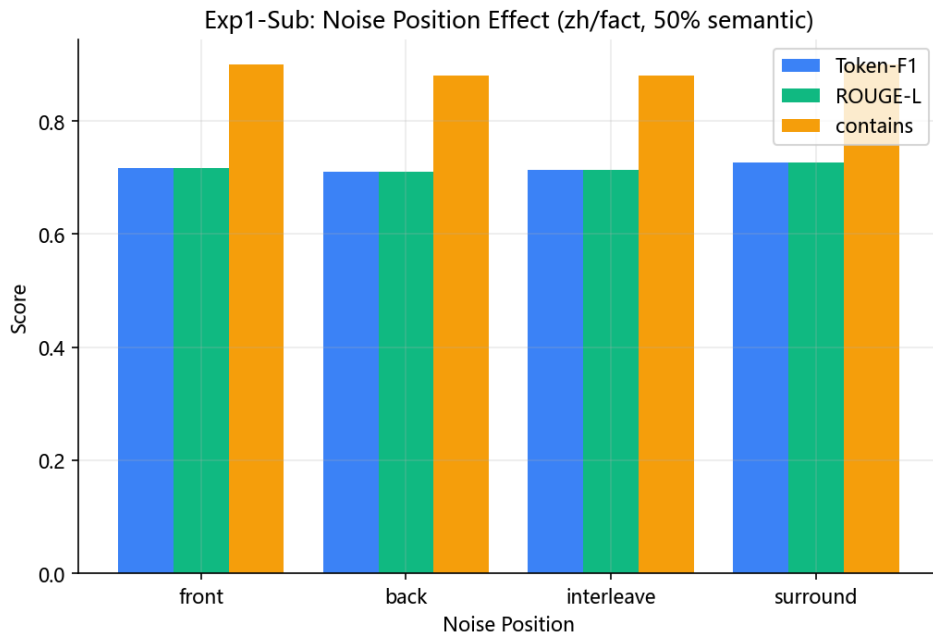


图 3: 噪音位置对问答指标的影响

在 50% 语义噪音下，不同位置策略的差异较小，当前不宜强行得出位置效应显著的结论。后续计划在 $r = 0.75$ 场景下复测，以观察更强噪音是否放大位置偏差。

7.3 矫正方法结果

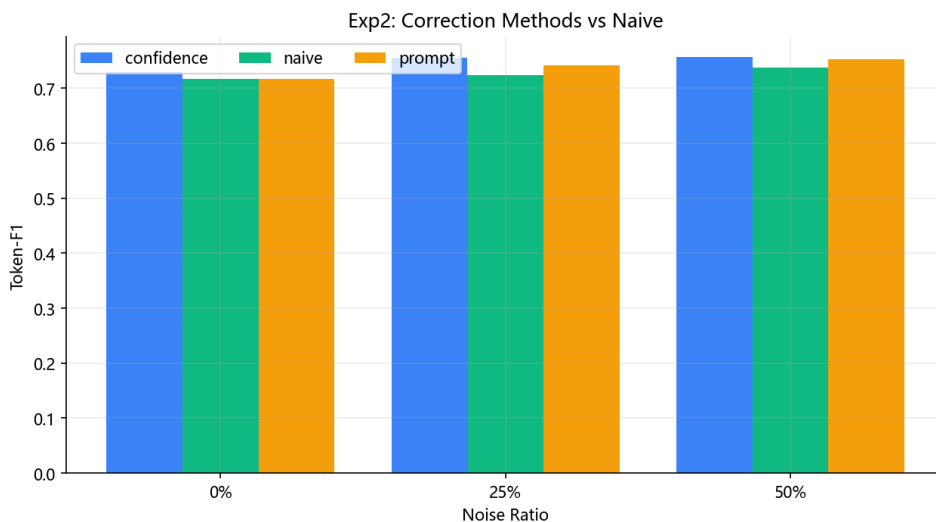


图 4: 实验二：矫正方法对比

中等语义噪音下，噪音不一定总是降低性能，部分噪音文档可能补充主题线索。因此正式结论需要避免简单写成“噪音越多越差”，而应区分噪音类型、比例和文档内容。

7.4 现有方法比较

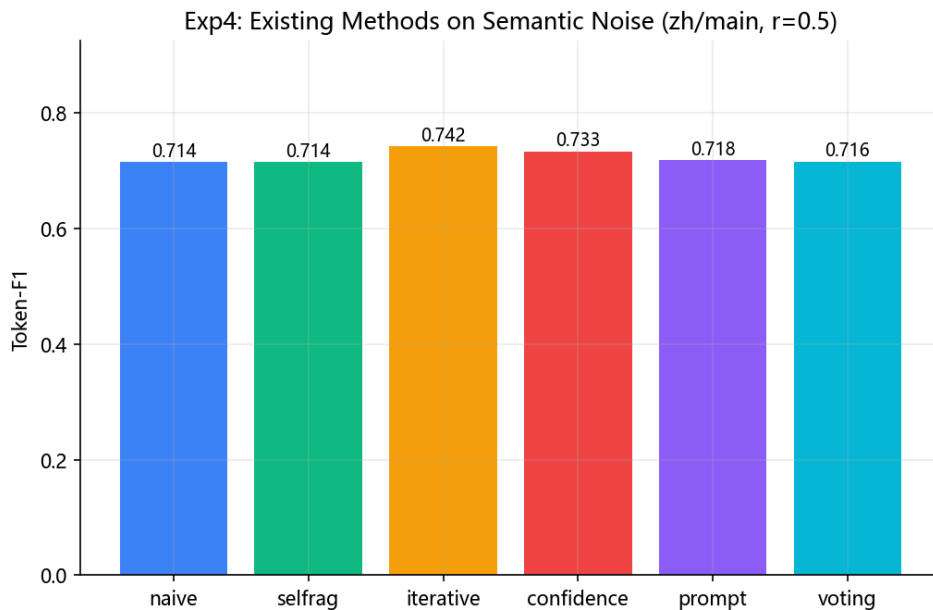


图 5: 实验四：六种方法横向对比，N=50

最新 exp4 结果如下：

表 5: zh/main semantic r=0.5 下六种方法表现

方法	EM	Contains	Token-F1	ISR	NAR
naive	0.38	0.88	0.7144	0.8205	0.0392
selfrag	0.38	0.88	0.7144	0.8205	0.0392
iterative	0.38	0.90	0.7415	0.8547	0.0152
confidence	0.36	0.90	0.7326	0.8046	0.0213
prompt	0.38	0.90	0.7178	0.8260	0.0278
voting	0.36	0.88	0.7158	0.8059	0.0274

当前结果支持：在该设置下 iterative 方法表现最好，说明先筛选证据再生成答案能减少噪音采纳。不过，exp4 当前只有单一噪音比例，NS/NRS/CRR 等跨比例鲁棒性指标仍需在后续扩展实验中补全。

7.5 案例类型分布

Exp3: Case Type Distribution

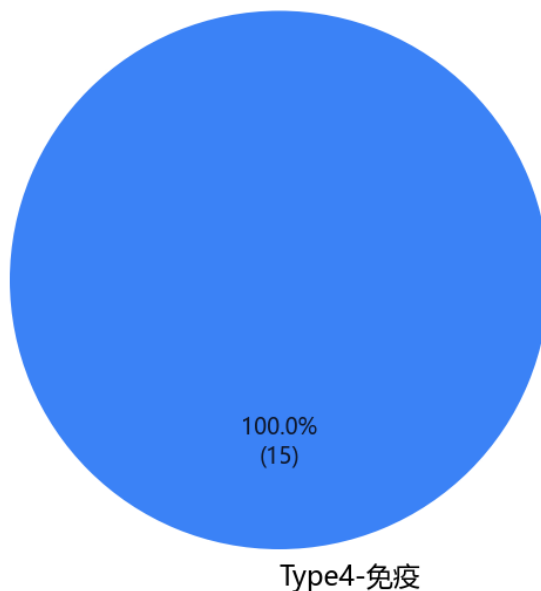


图 6: 实验三：案例类型分布

实验三已生成 15 个案例，case 中包含 query、gold、不同方法预测、ISR/NAR 和文档标签。后续正式报告将选取 3-5 个典型案例进行逐条分析。

8 系统实现与前端演示

8.1 后端接口

后端使用 FastAPI 实现，主要接口包括：

- /api/health: 健康检查；
- /api/samples: 读取样本；
- /api/sample/{id}: 查看单条样本；
- /api/experiment/inject: 构造噪音上下文；
- /api/experiment/run: 运行模型方法并返回指标。

8.2 前端界面

前端使用 Vue 3 + Vite，采用四阶段界面：

1. Stage A: 选择数据集和样本;
2. Stage B: 设置噪音比例、类型和位置;
3. Stage C: 选择方法并运行;
4. Stage D: 展示答案、指标和文档标签。

9 当前进度

表 6: 中期完成情况

模块	内容	状态
数据处理	RGB 8 个 JSONL 文件加载与标准化	已完成
噪音注入	比例、类型、位置三维控制	已完成
模型调用	DeepSeek API、缓存、重试、token 统计	已完成
RAG 基线	Naive RAG pipeline	已完成
矫正方法	prompt、iterative、confidence、selfrag、voting	已完成
评估指标	EM、Contains、Token-F1、ROUGE-L、 NS/NRS/ISR/NAR/CRR	已完成
实验脚本	exp1-exp4 与通用 runner	已完成
可视化	折线图、柱状图、雷达图、案例饼图	已完成
Web 系统	FastAPI 后端 + Vue 前端	已完成
标准交付	input/output/reference JSON 导出	已完成

10 问题、边界与后续计划

10.1 已识别问题

- 当前 exp4 仍是单比例比较，无法充分支撑跨比例鲁棒性结论;
- Self-RAG-style baseline 在当前样本上效果接近 naive，需要分析判定过宽的原因;
- exp2 样本量仍为 $N=30$ ，最终报告建议扩展到 $N=100$ 以上;
- 当前系统重点是受控上下文扰动实验，不是完整向量数据库检索系统，正式报告需要准确表述边界。

10.2 后续计划

表 7: 后续安排

阶段	任务	状态
W1	完成代码基础设施、首轮真实实验与中期报告	已完成
W2	扩展 N=50, 补跑 exp4 全方法与案例生成	已完成
W3	扩展到 N=100–300, 补充 counterfactual 与英文数据	计划中
W4	深化案例分析, 形成成功/失败/无变化分类	计划中
W5	调优 prompt 与矫正方法, 增加 bootstrap 置信区间	计划中
W6	整理最终报告、系统演示与提交压缩包	计划中

11 运行方式

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 配置 API
copy .env.example .env
# 编辑 .env, 填入 DEEPSEEK_API_KEY

# 3. 运行测试
set PYTEST_DISABLE_PLUGIN_AUTOLOAD=1
python -m pytest -q

# 4. 运行实验
python -m experiments.exp1_noise_impact --n 50 --language zh
python -m experiments.exp2_correction --n 50 --language zh
python -m experiments.exp3_case_study --n 30 --pick 15 --language zh
python -m experiments.exp4_existing_methods --n 50 --language zh --subset main --
    ratio 0.5 --noise-type semantic

# 5. 导出标准 JSON
python scripts/export_standard_io.py

# 6. 生成图表
python scripts/render_all_figures.py

# 7. 启动系统
```

```
uvicorn backend.main:app --reload --port 8000  
cd frontend && npm run dev
```

12 中期总结

本项目已完成从数据、方法、指标、实验到前端演示的主体闭环。当前最稳妥的阶段性结论是：高比例噪音会显著破坏 RAG 问答，ISR/NAR 能刻画答案来源从有效文档向噪音文档迁移；在当前 zh/main semantic r=0.5 设置下，iterative filter-then-generate 方法表现最好，但对“矫正方法整体显著优于 naive”的结论仍需更多噪音类型、更多比例和更大样本量验证。